EXPERIMENT 14

Exploring Pro-Code Extension of a Zoho Creator Application Using Zoho Catalyst and Custom Functions

---

Project: Insurance Claim Management System

Name	Pavithra Narayanan
Roll No	241401064
Department	CSBS

241401064

1. Aim

To explore how the Insurance Claim Management System built in Zoho Creator can be extended beyond low-code, using reusable Deluge custom functions and a pro-code serverless function on Zoho Catalyst, so that logic too complex for simple workflow actions can still be added to the application.

The exploration uses a claim priority scoring example to show how the two parts connect, when each option should be used, and what care is needed in a real implementation.

2. Procedure

Step 1: Identify the extension need — The existing Insurance Claim Request Form and the workflows from Experiments 6, 7 and 10 were reviewed. A new requirement was chosen that built-in actions handle poorly: calculating a priority level for each claim from the claim amount and the claim type.

Step 2: Compare the extension options — Workflow actions, Deluge custom functions, a Creator Custom API and a Catalyst function were compared on skill needed, power and typical use (Section 3.2).

Step 3: Design the architecture — The Creator side (form, workflow, custom function, connection) and the Catalyst side (Advanced I/O function and optional storage) were laid out in an architecture diagram (Section 3.1).

Step 4: Write the Deluge custom function — A reusable standalone function was drafted that builds a JSON payload and calls the Catalyst endpoint through a connection using invokeurl (Section 3.3).

Step 5: Write the Catalyst function — A Node.js Advanced I/O function was drafted that receives the payload, applies the scoring rules and returns the priority as JSON (Section 3.4).

Step 6: Plan the execution flow — The order of events from form submission to record update, including the error path, was drawn as a flowchart (Section 3.5), and a decision guide for choosing the extension type was prepared (Section 3.6).

Step 7: Note the precautions — Security, error handling, limits and maintenance points for such an integration were listed (Section 3.7).

24140164

Note: this is an exploration of the approach. The code shown is illustrative and was not deployed or executed in this record, and the endpoint and connection names are placeholders. A Priority_Level field is assumed to exist on the claim form for storing the result.

24140164

3. Output

3.1 Architecture of the pro-code extension

Zoho Creator keeps the form and workflow. On a successful submission the workflow calls a Deluge custom function, which sends an HTTPS request through a connection to an Advanced I/O function on Zoho Catalyst. The Catalyst function applies the business logic, optionally uses its own storage or an external API, and returns a JSON response that is used to update the record.

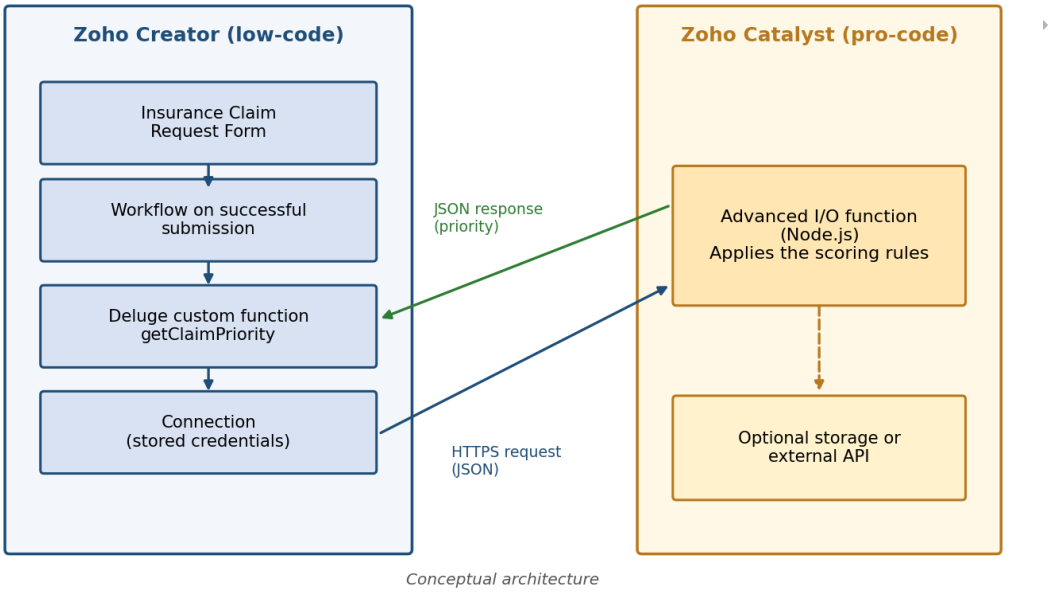


Figure 1: Architecture of the Pro-Code Extension (Conceptual Diagram)

3.2 Comparison of extension options

Option	What it is	Skill needed	Best used for
Workflow actions	Built-in actions such as notification, add or update record.	Low (no code)	Standard, simple automation.
Deluge custom function	Reusable standalone function written in Deluge and called from workflows.	Low to medium	Reusable business rules inside Creator.
Creator Custom API	Own REST endpoint exposed by the Creator application.	Medium	Letting other systems call into Creator.
Catalyst function	Serverless function in Node.js, Java or Python running outside Creator, called over HTTPS.	Medium to high (pro-code)	Complex logic, libraries, heavy processing, outside systems.

3.3 Deluge custom function (illustrative)

```
// Standalone function: getClaimPriority
string getClaimPriority(decimal amount, string claimType)
{
    payload = Map();
    payload.put("amount", amount);
    payload.put("claimType", claimType);
    response = invokeurl
    [
        url :"<Catalyst Advanced I/O endpoint>/score"
        type :POST
        parameters :payload.toString()
        headers :{"Content-Type":"application/json"}
        connection :"catalyst_connection"
    ];
    return response.get("priority");
}

// Called from the claim workflow (on success)
priority = thisapp.getClaimPriority(input.Claim_Amount_Requested,
input.Claim_Type);
Claim_Request_Form[ID == input.ID].Priority_Level = priority;
```

The custom function packages the request once so any workflow can reuse it. The workflow passes the claim amount and claim type, receives the priority and stores it, using the same criteria-based fetch with ID == input.ID as in Experiment 10. The URL and connection name are placeholders.

3.4 Catalyst Advanced I/O function in Node.js (illustrative)

```
const express = require("express");
const app = express();
app.use(express.json());

app.post("/score", (req, res) => {
    const { amount, claimType } = req.body;
    let score = 0;
    if (amount >= 500000) score += 3;
    else if (amount >= 100000) score += 1;
    if (claimType === "Health Insurance") score += 1;
    const priority = score >= 3 ? "High" : score >= 1 ? "Medium" : "Normal";
    res.status(200).json({ priority });
});

module.exports = app;
```

The function reads the JSON body, applies simple scoring rules and returns the priority. The rules and amounts are only an example for demonstration; real claim-priority rules would have to be defined and approved by the claims department.

3.5 Execution flow of the extended workflow

The flowchart shows the path from form submission to record update. If the response is not received successfully, the error is logged or an informational message is shown and the default priority is kept, so the claim is never lost because of the extension.

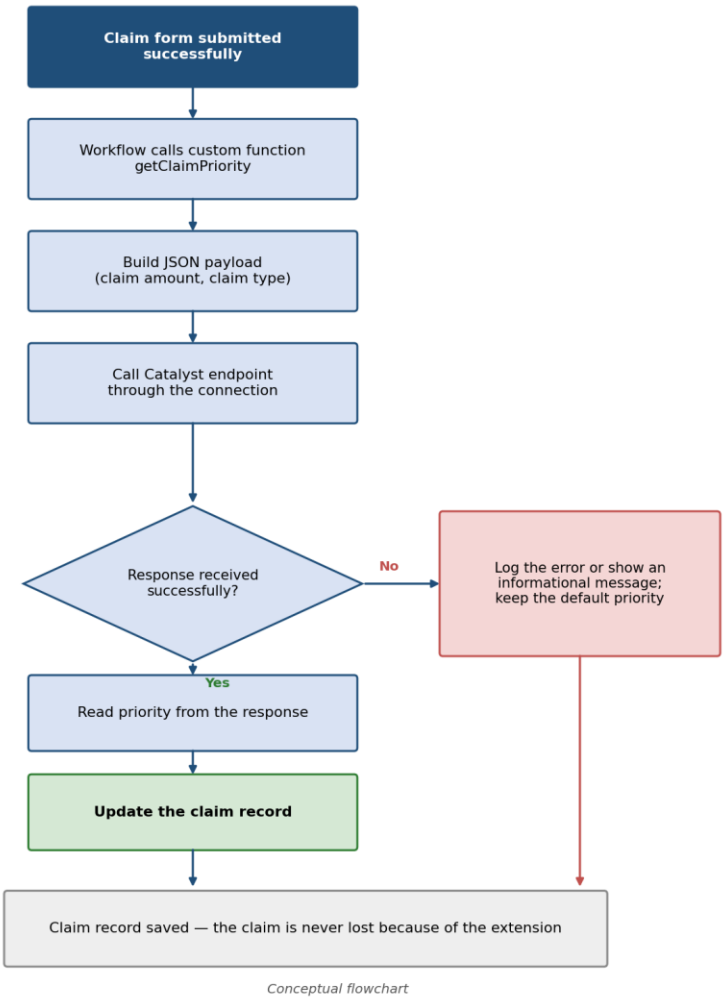


Figure 2: Execution Flow of the Extended Workflow (Conceptual Flowchart)

3.6 Decision guide for choosing the extension type

The guide first prefers built-in workflow actions, then a Deluge custom function for reusable Creator logic, and uses a Catalyst function only when libraries, heavy computation or outside systems are needed. If none apply, the requirement should be simplified.

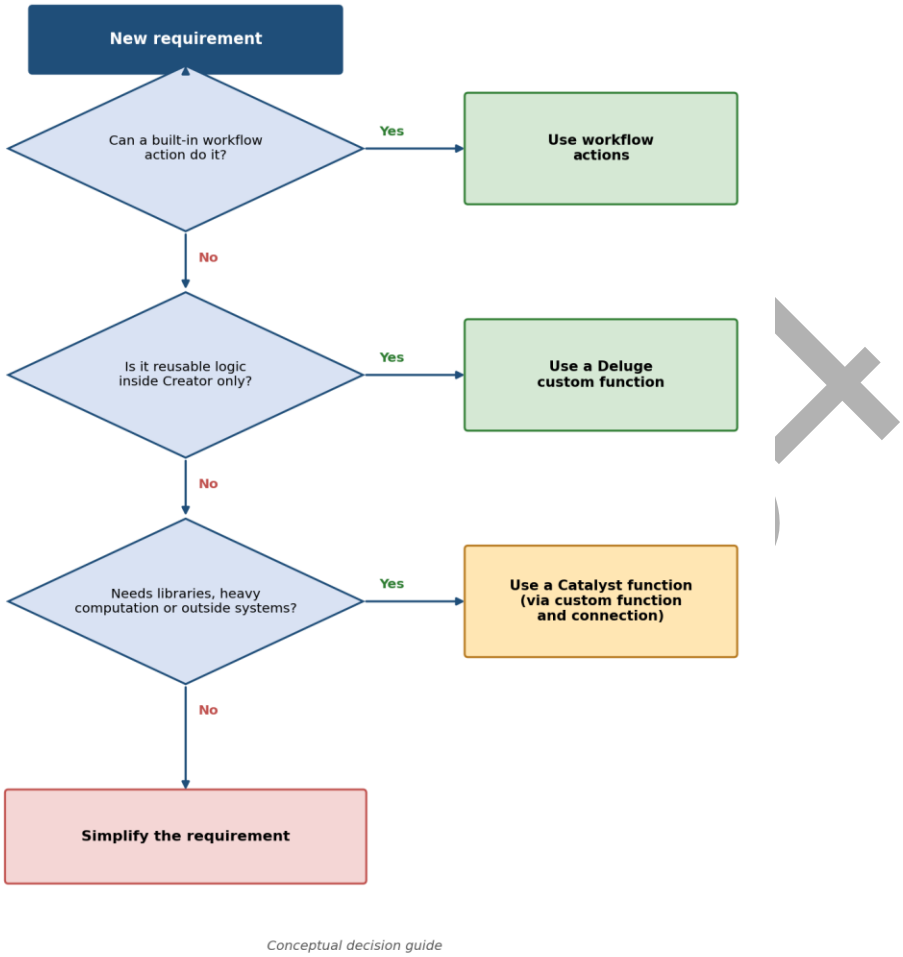


Figure 3: Decision Guide for Choosing the Extension Type (Conceptual Diagram)

3.7 Benefits and precautions

Aspect	Benefit	Precaution
Flexibility	Any logic or library can be used in a language such as Node.js, Java or Python.	More code to write, test and maintain than low-code.
Reuse	A custom function is written once and called from many workflows.	Keep the function interface (inputs and outputs) stable.
Separation	Heavy work runs outside Creator, keeping the application light.	Adds a network call, so add timeouts and error handling.
Security	Credentials stay in a connection instead of the script.	Do not put keys in code; restrict access and validate input.

Aspect	Benefit	Precaution
Limits and cost	Serverless scaling handled by the platform.	Check current execution limits and pricing in the documentation.

4. Final Result

The exploration showed how a low-code Zoho Creator application can be extended in two steps: a reusable Deluge custom function inside Creator, and a pro-code Zoho Catalyst function reached over HTTPS through a connection. The architecture, illustrative code, execution flow and decision guide were prepared for a priority scoring extension of the Insurance Claim Management System.

The comparison showed that workflow actions and custom functions are enough for most needs, and that Catalyst should be added only for complex logic, libraries or outside systems, with attention to security, error handling and limits.

The exercise met its objective of exploring pro-code extension. The code was not deployed or tested in this record, so results of a live run are not claimed.

24140106